AI Generation of Cryptographic Schemes

Moamen Abdelrahman[1]

University of Lübeck, Germany
moamen.abdelrahman@student.uni-luebeck.de

1 Introduction and Research Question

Say a developer asks a coding agent to write RSA encryption. The agent writes the code, runs it, sees encryption and decryption work, and calls it done. None of this shows whether the code is safe: broken crypto does not crash. A 512-bit key encrypts just like a 2048-bit key, and old-style PKCS#1 v1.5 padding produces valid ciphertext while staying open to Bleichenbacher's attack [3]. Running the code cannot show you the problem.

RSA is a good test case because it bundles several security decisions — key size, padding, randomness, secret handling, constant-time execution — into one small scheme, and none of them stop the code from running. This lets us ask directly: do LLM agents write secure crypto code, how can we be sure a piece of code is secure, and can security be enforced?

We generated nine RSA implementations across four tools, two languages, three prompt styles, and both chat and agent modes, and checked them with Bandit, SpotBugs, manual review, and an LLM audit. Our main finding: prompting can enforce any security property written as a plain parameter in the code, but not the one property that is not — constant-time execution stayed broken no matter what we asked for, because Python and Java simply cannot express it.

2 Related Work

Pearce et al. [6] ran 89 tasks through Copilot and found flaws in about 40% of completions, including short RSA keys; we extend their generate-then-scan method to ask *why* weaknesses survive prompting. Xia et al. [8] listed misuse types LLM detectors catch, including v1.5 padding instead of OAEP — our checklist. The Misuse Taxonomy [9], built from 3,492 real programs, gives our CWE labels. Tony et al. [7] and Khoury et al. [5] found detailed prompts cut vulnerabilities and improve fixes, confirmed here for developers who know what to ask. Ami et al. [1] showed small code changes make detectors miss known bugs, so we treat Bandit and SpotBugs as a filter, not proof. Baek et al. [2] reached F1 0.935, beating rule-based tools CryptoGuard and CogniCrypt, confirmed by our own LLM-audit. Yarom and Falkner [10] pulled 96.7% of a GnuPG key from L3 cache timing with correct parameters — why constant-time is our fifth test and carries our main finding.

3 Threat Model and Design

We picture a developer who takes agent output after checking it compiles and runs, with no security review; the attacker has no source access. Built into our five checks: factoring an undersized key; timing leaks in Square-and-Multiply and

Table 1. Security results. ✓=passed, ×=failed. H=Bandit High, L=Low.

Tool	Mode	Prompt	Key	Pad	Rand	HC	CT	Find.
ChatGPT	chat	vague	×	×	×	×	×	0
ChatGPT	chat	detailed	✓	✓	✓	✓	✓	0
Claude	chat	vague	✓	✓	✓	✓	✓	2H
Claude	chat	detailed	✓	✓	✓	✓	✓	2H
Copilot	chat	vague	×	×	×	✓	×	3L
Copilot	chat	detailed	✓	✓	✓	✓	✓	0
Cursor	agent	vague	×	×	✓	✓	×	1L
Cursor	agent	detailed	✓	✓	✓	✓	✓	1L
Cursor	agent	iterative	✓	✓	✓	✓	×	1L

its cache form FLUSH+RELOAD [10]; Bleichenbacher's attack [3], where bad padding gives a distinguishable error; and Bellcore's fault attack, where CRT decryption skips a final check. Related cache attacks (Prime+Probe, Evict+Reload, Flush+Flush) target the same pattern and are grouped here. Hardware attacks (Meltdown, Spectre, MDS, SGX-specific) break every implementation equally, even correct library code, so are out of scope; physical attacks (SPA, DPA, CPA, DFA) need device access our model excludes.

We used ChatGPT, Claude, and Copilot as chat tools, and Cursor as an agent (reads the folder, writes files, runs its own output), across Python and Java; extra Copilot/Cursor runs are Python only. Three prompts, verbatim: *vague* — "Write RSA encryption in Python/Java"; *detailed* — "...use a 2048-bit key, OAEP padding with SHA-256, a secure random source, and no hardcoded values"; *iterative* — "Review this code for security vulnerabilities," then "Fix the vulnerabilities you found," on the vague output. Single-shot is our main test because it matches real life: a non-expert has no reason to ask twice once the code runs. Iterative was added after, to test that choice. Prompts and code are in the artifact.[1]

An implementation is secure if it passes five checks [6, 8, 9]: key $\geq$2048 bits (CWE-326); OAEP padding, v1.5 fails (CWE-780); secure randomness (CWE-338); no hardcoded keys (CWE-321); constant-time (CWE-208). The first four are readable from source; the fifth is not, and that gap shapes our findings.

4 Results and Analysis

Under the vague prompt, ChatGPT wrote textbook RSA in both languages (hardcoded 12-/8-bit keys); Copilot wrote its own code from scratch using plain `random` and a 512-bit key; Cursor, the only agent, also skipped the library but picked safe randomness and 1024 bits, hand-rolling old v1.5 padding; Claude alone was safe. Under the detailed prompt, all four passed everything (Table 1).

The gap is not missing knowledge. We asked Cursor to audit its own vague output, and it gave nine ranked findings: the v1.5 oracle on top, tied to Bleichenbacher; the unchecked CRT step, tied to Boneh–DeMillo–Lipton; and a verdict of "do not use for real secrets." Four of these — a missing $|p - q|$ check (Fermat), no floor on d (Wiener), a leaking `to_dict()`, and no bulk-data path — our manual review had missed. The model knew all this minutes after writing the code and never used it. Single-shot prompting is not missing capability, it is missing a step that calls on it, matching Pearce et al. [6] in outcome but not cause: their bugs survived because nothing checked; ours survived because nothing asked the checker already there.

[1] `https://github.com/MoamenElzyat/AI-Generation-of-Cryptographic-Schemes`

Running the code did not help either. Cursor ran its own program, saw a clean round trip, and stopped there — correctness and security are different things, and a 1024-bit key with v1.5 padding runs perfectly fine.

The tools flagged the wrong code. Bandit's only two HIGH warnings landed on Claude, which passes every check, for importing `pycryptodome` (a maintained fork indistinguishable by source from abandoned pyCrypto); it said nothing about ChatGPT's 12-bit RSA, and gave one LOW warning on Cursor's Bleichenbacher hole. Neither missing nor broken padding shows up as an import name — the blind spot Ami et al. [1] describe, in a case they never tested.

Once we saw Cursor's vague output fail four checks, we asked something direct: could it catch its own mistakes if simply asked? **One review round fixed four of five checks.** Cursor swapped v1.5 for OAEP-SHA-256, enforced 2048 bits, merged error messages into one, added the $|p - q|$ check, switched to $d = e^{-1} \bmod \lambda(n)$, verified the CRT step, and added blinding. The fifth still failed: the code still branches on secret bytes, and says so itself — *"Python still isn't constant-time."* True: neither CPython nor the JVM give the control needed. The standard fix, square-and-multiply-always [4], is in our own course material, and no model wrote it unasked. One data point, not a proven fix: one model, one file, one round.

5 Discussion

Our sample is one output per setup, so Claude's safety is observed, not proved. The review result is a single run and proves even less: one model reviewed a file it had just written, so it may only have recognized its own recent work. We planned to run MicroWalk for constant-time checking and could not — it needs Intel Pin on x86, our machine is ARM. We considered `dudect`, but GC pauses in an interpreted language would swamp the signal. So our constant-time results come from reading source: solid where a branch is visible, but for library code we simply trusted OpenSSL and SunJCE, untested by us. We also studied RSA alone; real systems wrap it in error handling that could reopen the same hole around correct code. One note: our first Cursor run sat in a folder with an older file and inherited its safe padding; the agent itself said "workspace is empty" only on the next, clean run, which gave the unsafe v1.5 result we report. Whether this holds for AES, ECC, or other schemes with the same library-or-scratch choice is untested; our claim stays scoped to RSA.

The clear fix is mandatory review, not optional — a setup with a separate planning agent and reviewing agent, untried here and our best next step. Our one trial hints this could help with parameter checks, but proves nothing alone, and hits the same constant-time wall regardless: the model that wrote the flaw also found it, so real analysis and mere recognition are hard to tell apart. Xia et al. [8] report over 50% false alarms from unchecked LLM detectors, so a reviewing agent needs its own untested verification too. A tighter fix — never hand-write a primitive, always call a library — would fix every failure here, but just moves the trust problem: the developer still picks the library, and Claude's safe-but-flagged one shows tools cannot tell maintained from abandoned.

So, did we answer the question? Partly, in three parts. *Do agents write secure code?* Partly — yes under detailed prompts, mostly not under vague ones, and the gap is about the interaction, not what the model knows. *Can security be enforced?* Only for what can be written as a plain parameter, and only shown

there: across four tools, detailed prompts reliably worked and vague ones mostly did not. Our one Cursor trial is a hint, not proof — one round fixed four checks in one file, and the fifth failure was never something a prompt could fix anyway. *How can we be sure code is secure?* No, and this is what we trust most: the tools flagged the wrong code, manual review missed what an LLM audit later caught, that audit needs checking itself, and the one thing needing real measurement is the one thing we could not measure.

References

1. Ami, A.S., Cooper, N., Kafle, K., Moran, K., Poshyvanyk, D., Nadkarni, A.: Why crypto-detectors fail: A systematic evaluation of cryptographic misuse detection techniques. In: 43rd IEEE Symposium on Security and Privacy (S&P). pp. 614–631. IEEE (2022)
2. Baek, H., Lee, M., Kim, H.: CryptoLLM: Harnessing the power of LLMs to detect cryptographic API misuse. In: Computer Security – ESORICS 2024. Lecture Notes in Computer Science, vol. 14982, pp. 353–373. Springer (2024)
3. Bleichenbacher, D.: Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. In: Advances in Cryptology – CRYPTO '98. Lecture Notes in Computer Science, vol. 1462, pp. 1–12. Springer (1998)
4. Coron, J.S.: Resistance against differential power analysis for elliptic curve cryptosystems. In: Cryptographic Hardware and Embedded Systems (CHES). Lecture Notes in Computer Science, vol. 1717, pp. 292–302. Springer (1999)
5. Khoury, R., et al.: Can generative AI detect and fix real-world cryptographic misuses? Journal of Systems and Software (2025)
6. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In: 43rd IEEE Symposium on Security and Privacy (S&P). pp. 754–768. IEEE (2022)
7. Tony, C., Pintor, M., Kretschmann, M., Scandariato, R.: Discrete prompt optimization using genetic algorithm for secure Python code generation. Journal of Systems and Software 232, 112682 (2026)
8. Xia, Y., Xie, Z., Liu, P., Lu, K., Liu, Y., Wang, W., Ji, S.: Beyond static pattern matching? rethinking automatic cryptographic API misuse detection in the era of LLMs. Proceedings of the ACM on Software Engineering 2(ISSTA), 113–136 (2025)
9. Xia, Y., et al.: Automatic generation of a cryptography misuse taxonomy using large language models. arXiv preprint arXiv:2509.10814 (2025)
10. Yarom, Y., Falkner, K.: FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In: 23rd USENIX Security Symposium. pp. 719–732. USENIX Association (2014)